

บทที่ 2

เชิร์ฟเฟลิต

2.1 จาวาเชิร์ฟเฟลิต

จาวาเชิร์ฟเฟลิต คือเทคโนโลยี ที่เกิดขึ้นมาจากจาวา เพื่อใช้งานทางด้านการสร้างโปรแกรมในรูปแบบซีจีไอสำหรับทำงานบนเว็บเซิร์ฟเวอร์ เพื่อให้ผลการแสดงเว็บสู่ผู้อ่าน มีการเปลี่ยนแปลงตามการใช้งาน (On The Fly) ซึ่งเป็นเทคโนโลยีที่สืบเนื่องมาจากเหตุผลดังนี้

- ความต้องการให้หน้าเว็บเปลี่ยนตามข้อมูลที่ผู้อ่านส่งให้ เช่น เว็บที่ให้บริการระบบค้นหา (Search Engine)ทั้งหลาย ที่ได้รับข้อมูลจากผู้ใช้ คือข้อความที่ต้องการค้นหา และแสดงผลลัพธ์หน้าเว็บ ตามข้อมูลที่ค้นหาเจอแก่ผู้ใช้
- ความต้องการให้หน้าเว็บเปลี่ยนตามข้อมูลที่มีการเปลี่ยนแปลงบ่อยครั้ง เช่น เว็บที่เกี่ยวข้องกับการรายงานข่าว หรือสภาพอากาศ จะเห็นว่าข้อมูลที่รายงานสู่ผู้ใช้ ดังกล่าว มีการเปลี่ยนแปลงอยู่ตลอดเวลา อาจเป็นรายชั่วโมง หรือต่อนาทีก็ได้
- ความต้องการให้หน้าเว็บเปลี่ยนตามระบบฐานข้อมูล เช่น ระบบควบคุมคลังสินค้า ที่อาศัยการเก็บข้อมูลในระบบฐานข้อมูล สามารถแสดงผลจำนวนสินค้า ตามข้อมูลที่มีอยู่จริงในฐานข้อมูล ออกสู่ผู้ใช้บนระบบเว็บ

2.2 ข้อแตกต่างของเชิร์ฟเฟลิต เมื่อเทียบกับ ซีจีไอ

เชิร์ฟเฟลิต เป็นเทคโนโลยี ที่มีรูปแบบการทำงานเหมือนกับ เทคโนโลยีซีจีไอ ที่มีการใช้งานอยู่ แต่เนื่องจาก เชิร์ฟเฟลิต ถูกพัฒนามาจากเทคโนโลยีจาวาอีกที ดังนั้น ต่อไปนี้คือข้อเปรียบเทียบซึ่งแสดงให้เห็น ถึงความแตกต่างระหว่างเชิร์ฟเฟลิตและซีจีไอ

- ประสิทธิภาพ การทำงานของซีจีไอจะมีการสร้างโปรเซสใหม่ทุกครั้งที่มีการเรียกใช้งานจากผู้ใช้ ดังนั้นหากมีการเรียกใช้งานจำนวนหลายครั้งในช่วงเวลาเดียวกัน ก็จะมีจำนวนโปรเซสเกิดขึ้นเท่ากับจำนวนครั้งที่เรียกใช้งาน โปรเซสจะมีขนาดและรูปแบบการทำงานที่เหมือนกันทำให้กินเวลาในการทำงานตามจำนวนโปรเซส แต่สำหรับเชิร์ฟเฟลิตแล้วระบบเวอร์ชวลแมชีน จะถูกดึงขึ้นมาทำงานก่อน เพื่อทำให้เกิดโปรเซสตามลักษณะการเรียกใช้ของผู้ใช้เช่นกัน แต่การทำงานของเชิร์ฟเฟลิตจะเป็นรูปแบบเธรด(Thread) แทนที่จะเรียกว่าโปรเซส ซึ่งแต่ละเธรดขนาดการทำงานเล็กกว่าเดิม และกินระยะเวลาการทำงานน้อยกว่า จึงทำให้รูปแบบของเชิร์ฟเฟลิต มีประสิทธิภาพกว่านั่นเอง
- ความสะดวก หากคุณสามารถเขียนโปรแกรม ด้วยภาษาซีหรือ จาวาอยู่แล้ว จะเกิดความสะดวก และรวดเร็วในการสร้างเชิร์ฟเฟลิตมากกว่า แทนที่จะเริ่มศึกษาภาษาคำสั่งใหม่ๆ และเชิร์ฟเฟลิตก็มีความสามารถเพียงพอสำหรับการใช้งานบนเว็บอยู่แล้ว เช่น การรับข้อมูลจากฟอร์ม, การอ่านและตั้งค่า HTTP Header, การทำงานกับคุกกี้, การทำงานแบบเซสชัน และอื่นๆอีกมาก

- ความสามารถเซิร์ฟเล็ทมีข้อดีที่เห็นได้ชัดคือ สามารถติดต่อกับเว็บเซิร์ฟเวอร์ได้โดยตรง ในขณะที่ ซีจีไอ ปกติทำไม่ได้ (ยกเว้นรุ่นใหม่ๆ) และเซิร์ฟเล็ทยังสามารถแชร์ข้อมูลการทำงาน ร่วมกับโปรแกรมอื่นๆ ได้ด้วย
- การโยกย้าย เนื่องจากเซิร์ฟเล็ทถูกสร้างมาจากจาวา ซึ่งเป็นเทคโนโลยี ที่สามารถถูกใช้งาน ณ. ที่ใดก็ได้โดยไม่จำกัดรูปแบบ ดังนั้น หากพัฒนาเซิร์ฟเล็ทเพื่อใช้งานแล้วสามารถที่จะโยกย้าย ผลผลิตไปยัง ระบบอื่นๆ ได้ โดยไม่ต้องปรับปรุงหรือเปลี่ยนแปลง
- ไม่แพง ในปัจจุบันเว็บเซิร์ฟเวอร์ที่สนับสนุนการทำงานเซิร์ฟเล็ท มีอยู่มากมาย ทั้งที่เป็นแบบฟรีดาวน์โหลด (Download) และซื้อ-ขายกัน นอกจากนี้ในอนาคตแผนการผลิตเว็บเซิร์ฟเวอร์อื่นๆ ยังมีการสนับสนุนการใช้งานเซิร์ฟเล็ทอีกด้วย

2.3 การอ่านข้อมูลจากเว็บ

สำหรับการใช้งานเว็บผ่านบราวเซอร์ โดยทั่วไปแล้ว ปกติในช่องที่ใช้กำหนดข้อมูลยูอาร์แอล (URL) จะประกอบด้วยชื่อเซิร์ฟเวอร์ ชื่อไดเรกทอรี และชื่อไฟล์เว็บเพจที่ต้องการดู เช่น หากข้อมูลยูอาร์แอล คือ <http://www.javacentrix.com/tutorials/api/index.htm> นั้นหมายถึงต้องการอ่านเว็บเพจที่ชื่อ index.htm จากไดเรกทอรี tutorials/api จากเว็บเซิร์ฟเวอร์ที่ชื่อ www.javacentrix.com

สำหรับยูอาร์แอล นั้นนอกจาก จะใช้ตำแหน่งไฟล์ที่ต้องการเรียกดูแล้ว ยังสามารถใช้สำหรับส่งข้อมูลให้กับระบบเซิร์ฟเวอร์ เพื่อใช้ในการทำงาน หรือดำเนินการอย่างใดอย่างหนึ่ง ลักษณะยูอาร์แอล ประเภทนี้หลังจากชื่อไฟล์แล้วจะตามด้วยเครื่องหมายคำถาม ‘?’ และข้อมูลที่ต้องการส่งไปให้เซิร์ฟเวอร์ ซึ่งเรียกโดยทั่วไปว่าการส่งข้อมูลแบบ GET โดยมีรูปแบบคือ

URL?Parameter1=Data1&Parameter2=Data2

จากรูปแบบการส่งข้อมูลผ่านยูอาร์แอล โดย **Parameter1** และ **Parameter2** คือชื่อพารามิเตอร์ที่ถูกเซิร์ฟเวอร์อ้างอิงได้ ในขณะที่ Data1 และ Data2 คือข้อมูลที่อยู่ในพารามิเตอร์แต่ละตัว หากต้องการส่งข้อมูลมากกว่า 2 ตัว ก็จะเพิ่มต่อท้ายไปได้เรื่อยๆ เช่น

URL?Parameter1=Data1&Parameter2=Data2&Parameter3=Data3&Parameter4=Data4

ตัวอย่างเช่น

<http://www.javacentrix.com/index.htm?Name=JAVACENTRIX&Description=ThaiJavaWebSite>

หน้าเครื่องหมาย “?” คือ ข้อมูลยูอาร์แอล ในขณะที่ส่วนที่เหลือคือ ข้อมูลที่ส่งให้กับเซิร์ฟเวอร์ อธิบายได้ดังนี้คือ เป็นเรียกใช้งานไฟล์ index.htm โดยที่มีการส่งข้อมูลไปให้กับเซิร์ฟเวอร์ 2 ตัวคือ Name ที่มีข้อมูล "JAVACENTRIX" และ Description ที่มีข้อมูล "Thai Java Web Site" เซิร์ฟเวอร์จะนำข้อมูลเหล่านี้ไปใช้ประกอบการทำงานกับไฟล์ index.htm ซึ่งในที่นี้หมายถึง เซิร์ฟเวอร์นั่นเอง

หากผู้อ่านเคยใช้เว็บที่มีระบบค้นหาและพิจารณาในช่องยูอาร์แอล ก็จะสังเกตเห็นข้อมูลที่เป็น เช่น อธิบายไว้หลังจากคลิกที่ปุ่มค้นหา แต่ข้อเสียของการส่งข้อมูลรูปแบบนี้ คือ ผู้ใช้ต้องพิมพ์ชื่อพารามิเตอร์ ให้ถูก และพิมพ์ข้อมูลให้ถูกรูปแบบด้วย อีกทั้งการใช้งานยังไม่เป็นความลับ ซึ่งถูกมองเห็นได้จากคนทั่วไป

นอกจากผู้ใช้เว็บจะต้องพิมพ์ข้อมูลที่ยูอาร์แอล เองแล้ว การส่งข้อมูลแบบ **GET** นี้ยังสามารถใช้แท็ก **FORM** ในเอชทีเอ็มแอล เพื่อสร้างรูปแบบการกรอกข้อมูลแก่ผู้ใช้ ที่เรียกว่าฟอร์ม เมื่อผู้ใช้ข้อมูลกรอกข้อมูลผ่านฟอร์ม และกดปุ่ม **Submit** ข้อมูลเหล่านั้นจะถูกบราวเซอร์ ประกอบเป็นยูอาร์แอล ในรูปแบบ **GET** ให้อย่างอัตโนมัติ โดยที่ผู้อ่านไม่จำเป็นต้องพิมพ์ยูอาร์แอลเองให้เสียเวลาเลย

ในทำนองเดียวกัน การส่งข้อมูลจากผู้ใช้ นอกจากจะส่งในรูปแบบ **GET** แล้วยังมีอีกรูปแบบหนึ่งคือรูปแบบ **POST** ซึ่งแทนที่จะส่งไปที่ตัวยูอาร์แอลโดยตรง แต่เป็นการส่งด้วยโปรโตคอลเอชทีทีพี ที่ใช้ในเว็บเป็นตัวจัดการ การส่งแบบนี้มีข้อดีที่ผู้อื่น ไม่สามารถล่วงรู้ได้ว่าลักษณะการส่งข้อมูลไปใช้เซิร์ฟเวอร์ ส่งข้อมูลอะไรไปและส่งโดยการใช้อุปกรณ์อะไร การส่งข้อมูลในแบบ **POST** ทำได้โดยการใช้แท็ก **FORM** แต่กำหนดวิธีการส่งเป็น **POST** นอกนั้นการสร้างฟอร์มก็ใช้การสร้างยังส่งข้อมูลแบบ **GET** ได้ด้วย หากสร้างฟอร์มด้วยการใช้แท็ก **FORM** แต่กำหนดวิธีการส่งเป็น **GET**

การสร้างฟอร์มแบบ GET และ POST

```
<HTML><HEAD><TITLE>Test Form </TITLE></HEAD>
<BODY>
  <FORM method="get" action="myParameter">
    ชื่อ <input type="text" name="name"> <BR>
    นามสกุล <input type="text" name="surname"> <BR>
    ชาย<input type="radio" name="sex" value="M">
    หญิง<input type="radio" name="sex" value="F"> <BR>
    ที่อยู่ <textarea name="add"></textarea><br>
    E-mail <input type="text" name="mail"> <BR>
    <input type="submit" name="Submit" value="Submit">
  </FORM>
</BODY>
</HTML>
```

จากคำสั่งเซชที่เอนแอลที่เห็นจากตัวอย่าง มีช่องสำหรับให้ผู้ใช้กรอก 3 ช่องคือที่เก็บชื่อที่ชื่อ **name, surname** และ **mail** (สังเกตจากคำสั่งในแท็ก FORM ของ HTML) และมีการเรียกใช้รูปแบบการส่งข้อมูลแบบ GET (สังเกตจากในแท็กฟอร์ม แอททริบิวต์ `method="get"`) ผู้อ่านสามารถทดลองเปลี่ยนคำว่า `get` เป็นคำว่า `post` ได้หากต้องการส่งข้อมูลรูปแบบ POST จำไว้ว่าการส่งแบบ GET ข้อมูลจะถูกพร้อมกับยูอาร์แอล ในขณะที่ส่งแบบ POST ข้อมูลถูกส่งด้วยรูปแบบคำสั่งโปรโตคอลเอชทีทีพีแทน

2.4 สร้างเซิร์ฟเล็ตอ่านข้อมูลจากฟอร์ม

คำสั่งที่ใช้สำหรับอ่านข้อมูลจากผู้ใช้คือ `getParameter` ที่อยู่ในคลาส `HttpServletRequest`

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
import java.util.*;

public class ShowFormData extends HttpServlet
{ public void doGet(HttpServletRequest request, HttpServletResponse response)
  throws ServletException, IOException {
    response.setContentType("text/html");
    String title = "Retriving Information from Web-Form";
    PrintWriter out = response.getWriter();
    out.println("<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.0
Transitional//EN"> " +
        "<HTML> " +
        "<HEAD><TITLE>Hello WWW</TITLE></HEAD> " +
        "<BODY> " +
        "<H1>" + title + "</H1> ");
    out.print("<TABLE BGCOLOR=\"#EEEEEE\"?> ");
    out.print("<TR><TD align=\"center\">Parameter Name</TD> ");
    out.print("      <TD align=\"center\">Parameter Value(s)</TD></TR> ");
    Enumeration paramNames = request.getParameterNames();
    while(paramNames.hasMoreElements()) {
        String paramName = (String) paramNames.nextElement();
        out.print("<TR><TD valign=\"top\">" + paramName + "</TD> ");
```

```

String[] paramValues = request.getParameterValues(paramName);
if (paramValues.length == 1) {
    String paramValue = paramValues[0];
    if (paramValue.length() == 0)
        out.print("<TD>No Value</TD>");
    else
        out.print("<TD>" + paramValue + "</TD>");
} else {
    out.println("<TD>");
    for(int i=0; i<paramValues.length; i++)
        out.println(paramValues[i] + "<BR>");
    out.println("</TD>");
}
out.print("</TR> ");
}
out.println("</TABLE> " +
    "</BODY></HTML>");
}

public void doPost(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    doGet(request, response);
}
}

```

2.5 การใช้งานคุกกี้ด้วยเซิร์ฟเล็ต

ในการใช้งานเว็บพลิเคชันเบราว์เซอร์จะได้รับเนื้อข้อมูลซึ่งอาจจะอยู่ในรูปของข้อความ รูป, เสียง หรือภาพเคลื่อนไหวและข้อมูลอื่นๆ โดยที่เบราว์เซอร์สามารถจัดเก็บไว้ที่พื้นที่ที่เรียกว่าแคช เพื่อนำมาใช้ งานในภายหลังได้และข้อมูลดังกล่าวที่ใช้ในการจัดเก็บถือได้ว่าเป็นเนื้อข้อมูลที่เว็บเซิร์ฟเวอร์ต้องการที่จะ นำเสนอต่อเบราว์เซอร์เท่านั้น แต่หากเว็บเซิร์ฟเวอร์ต้องการส่งข้อมูลนอกเหนือจากนั้นมาจัดเก็บไว้ใน พื้นที่ใดๆของเบราว์เซอร์ก่อน เพื่อในภายหลังหากเบราว์เซอร์เรียกใช้งานเว็บเซิร์ฟเวอร์ตัวเดิม เว็บ เซิร์ฟเวอร์ก็จะหยิบเอาข้อมูลนี้มาใช้งานในภายหลัง ซึ่งข้อมูลนี้เรียกกันโดยทั่วไปว่าคุกกี้ การใช้ประโยชน์ จากคุกกี้สามารถนำมาประยุกต์การใช้งานได้ เช่น หากเว็บเซิร์ฟเวอร์ต้องการชื่อและนามสกุลผู้ใช้ใน ครั้งแรก และเบราว์เซอร์ได้บอกข้อมูลเหล่านี้ผ่านทางฟอร์มไปแล้ว เว็บเซิร์ฟเวอร์ก็จะส่งชื่อและนามสกุล

ผู้ใช้งานเก็บไว้ที่เบราว์เซอร์เอง และหลังจากนั้นหากเบราว์เซอร์มีการเรียกใช้เว็บเซิร์ฟเวอร์ตัวเดียวกันนี้อีก เว็บเซิร์ฟเวอร์ก็จะอ่านชื่อและนามสกุลที่ฝากไว้ที่เบราว์เซอร์มาใช้ได้เลย โดยที่ผู้ใช้เบราว์เซอร์ไม่จำเป็นต้องกรอกชื่อและนามสกุลเพื่อบอกให้เว็บเซิร์ฟเวอร์ทราบอีก ซึ่งเป็นการอำนวยความสะดวกในการใช้งานเกิดขึ้นนั่นเอง

เซิร์ฟเล็ตสามารถควบคุมการใช้งานคุกกี้ผ่านทางข้อมูลส่วนหัวของโปรโตคอลเอชทีทีพี โดยหากต้องการจัดเก็บข้อมูลคุกกี้ไว้ที่เบราว์เซอร์ก็กำหนดผ่านข้อมูลส่วนหัวได้ตอบ (ส่วน Set-Cookie) ไปยังเบราว์เซอร์ก่อน หลังจากนั้นหากเว็บเซิร์ฟเวอร์มีการอ้างจากเบราว์เซอร์ในเส้นทางเดิมอีกเบราว์เซอร์ก็จะส่งข้อมูลคุกกี้ที่มีข้อมูลส่วนหัวร้องขอ (ส่วนคุกกี้) มาให้เว็บเซิร์ฟเวอร์ด้วย ดังนั้นเซิร์ฟเล็ตก็สามารถอ่านข้อมูลดังกล่าวมาใช้งานได้ แต่สำหรับการใช้งานคุกกี้แล้วคลาส `HttpServletRequest` และ `HttpServletResponse` มีเมธอดเฉพาะสำหรับการดำเนินงานในส่วนคุกกี้ให้ใช้งานได้อย่างสะดวก โดยไม่จำเป็นต้องจัดการข้อมูลส่วนหัวให้ยุ่งยากเลย ดังมีรายละเอียดคือ

รายละเอียดที่เกี่ยวข้องกับการใช้งานคุกกี้	
<code>Cookie()</code>	คอนสตรักเตอร์ของคลาส <code>Cookie</code>
<code>Cookie(String name, String value)</code>	คอนสตรักเตอร์ของคลาส <code>Cookie</code> สำหรับกำหนดชื่อ <code>name</code> และข้อมูลใน <code>value</code> โดยการตั้งชื่อคุกกี้ นั้นให้ใช้ภาษาอังกฤษสามารถมีช่องว่าง และใช้สัญลักษณ์ <code>[] () = , " / ? @ : ;</code> ได้
เมธอดที่เกี่ยวข้องกับการกำหนดค่าและการอ่านค่าของคลาส <code>Cookie</code>	
<code>public void setComment(String comment)</code>	ใช้กำหนดข้อมูลเสริมของคุกกี้ เหมือนการใส่หมายเหตุเพื่ออธิบายรายละเอียดของการใช้งานคุกกี้ตัวนี้ โดยแต่ละคุกกี้สามารถมีข้อมูลหมายเหตุแยกแต่ละตัวได้
<code>public String getComment()</code>	สำหรับอ่านข้อมูลหมายเหตุของคุกกี้
<code>public void setDomain(String domain)</code>	สำหรับกำหนดชื่อโดเมน เพื่อให้ใช้งานคุกกี้เดียวกันนี้ได้ โดยรูปแบบโดเมนต้องมีสัญลักษณ์ <code>.</code> อยู่มากกว่า 2 ตำแหน่ง เช่น <code>.javacentrix.com</code> หรือ <code>.kmitl.ac.th</code> เป็นต้น
<code>public String getDomain()</code>	สำหรับอ่านค่าโดเมนเพื่อใช้งาน
<code>public void setMaxAge(int lifetime)</code>	สำหรับกำหนดอายุคุกกี้ที่จะเก็บไว้ที่เบราว์เซอร์ หากเกินกำหนดเบราว์เซอร์จะลบออกจากเครื่อง มีหน่วยเป็นวินาที หากกำหนดเป็น 0 หรือค่าติดลบ หมายถึงให้ลบทันทีที่เลิกใช้งานเบราว์เซอร์
<code>public int getMaxAge()</code>	สำหรับอ่านข้อมูลอายุของคุกกี้

public void setName(String name)	สำหรับกำหนดชื่อให้กับคุกกี้
public String getName()	สำหรับอ่านชื่อของคุกกี้
public void setPath(String path)	สำหรับกำหนดเส้นทางที่จะใช้งานคุกกี้ได้ โดยระบุเป็นข้อมูลยูอาร์แอล โดยที่ไม่ต้องระบุไฟล์เอกสาร เช่น http://online.javacentrix.com/order/ แต่ไม่ใช่ http://online.javacentrix.com/order/new.html
public String getPath()	สำหรับอ่านข้อมูลเส้นทางที่คุกกี้จัดเก็บไว้สำหรับใช้งาน
public void setSecure(boolean flag)	สำหรับค่าสถานะการรักษความปลอดภัยในการเข้ารหัสข้อมูลแบบ SSL คือเริ่มต้นคือ false
public boolean getSecure()	สำหรับค่าสถานะการรักษความปลอดภัย
public void setValue(String value)	สำหรับกำหนดข้อมูลให้กับคุกกี้
public String getValue()	สำหรับอ่านชื่อคุกกี้
public void setVersion(int version)	สำหรับกำหนดเวอร์ชันให้กับคุกกี้
public int getVersion()	สำหรับอ่านเวอร์ชันของคุกกี้

ตารางที่ 2-1 แสดงรายละเอียดที่เกี่ยวข้องกับการใช้งานคุกกี้

การส่งคุกกี้ไปเก็บที่เบราว์เซอร์ ในการส่งข้อมูลไปเก็บไว้เป็นคุกกี้ในเบราว์เซอร์จำเป็นต้องเริ่มต้นจากเซิร์ฟเวอร์ส่งไปให้ในครั้งแรกก่อน ซึ่งการจะเก็บได้หรือไม่ขึ้นอยู่กับความยินยอมที่กำหนดจากโปรแกรมเบราว์เซอร์เอง ซึ่งการจัดเก็บคุกกี้นี้หากเบราว์เซอร์ยินยอมแล้วผู้ใช้จะไม่สามารถสังเกตเห็นเลย ว่ามีการจัดเก็บข้อมูลคุกกี้หรือไม่จากการใช้งานปกติ (ยกเว้นผู้ใช้เข้าไปดูคุกกี้ภายในเครื่องใช้งานเอง) โดยต้องอาศัยเมธอด addCookie ของคลาส HttpServletResponse จัดเก็บดังนี้

การอ่านคุกกี้จากเบราว์เซอร์ สำหรับการอ่านข้อมูลคุกกี้ที่เซิร์ฟเวอร์ที่ทำหน้าที่อ่านข้อมูลจากคุกกี้ได้นั้น สามารถเรียกใช้เมธอด getCookie จากคลาส HttpServletRequest ได้เลย หากเคยส่งคุกกี้ไปเก็บไว้ที่เบราว์เซอร์แล้ว แต่หากไม่เคยเมธอดดังกล่าวจะคือค่า null กลับมา และเมื่อเซิร์ฟเวอร์ได้รับคุกกี้กลับมา ก็สามารถเรียกใช้เมธอดคุกกี้ในกลุ่ม getXxx ที่มีรายละเอียดอยู่ก่อนหน้าี่อ่านข้อมูลมาใช้งานได้ต่อไป

2.6 เจดีบีซี (JDBC) ช่องทางติดต่อฐานข้อมูลด้วยจาวา

จาวาด้าเบส คอนเนกทิวิตี (Java Database Connectivity) หรือ เจดีบีซี (JDBC) คือมิดเดิลแวร์ที่ถูกใช้บนเทคโนโลยีจาวา ในการเข้าถึงระบบฐานข้อมูล โดยสามารถเรียกใช้งานข้อมูล หรือจัดการระบบฐานข้อมูลผ่าน คำสั่ง SQL-92 หรือภาษาเอสคิวแอลมาตรฐาน

การใช้งานเจดีบีซี

อย่างที่ทราบกันว่าการเข้าใช้งานฐานข้อมูลด้วยภาษาจาวา มีรูปแบบอยู่ด้วยกันถึง 4 ชนิด การที่โปรแกรมเมอร์จะเลือกใช้วิธีใดก็ขึ้นอยู่กับความเหมาะสมของระบบว่าระบบถูกออกแบบมาให้มีลักษณะการทำงานแบบใด แต่ถึงแม้ว่าจะได้ข้อสรุปอย่างไรก็ตาม วิธีการลงมือเขียนโปรแกรมก็มีขั้นตอนที่คล้ายกัน ดังนี้

- ขั้นตอนเริ่มติดต่อกับฐานข้อมูล (Establish Database)
- ขั้นตอนการส่งคำสั่งเอสคิวแอลให้กับฐานข้อมูล (Send Query)
- ขั้นตอนการแสดงผลลัพธ์สู่ผู้ใช้ (Display Result)
- ขั้นตอนการปิดการติดต่อกับฐานข้อมูล (Close Connection)

ขั้นตอนที่ 1 เริ่มติดต่อกับฐานข้อมูล

แบ่งออกเป็น 2 ขั้นตอนย่อยๆดังนี้ คือ

A) ขั้นตอนการโหลดไดรเวอร์เพื่อใช้งาน

รูปแบบ

```
Class.forName ("Driver_Name")
```

คลาสดักจับข้อผิดพลาด

```
ClassNotFoundException
```

เมื่อ

Driver_Name คือชื่อไดรเวอร์สำหรับใช้งานระบบฐานข้อมูล ที่ต้องการโหลดขึ้นมาใช้งาน ซึ่งจะต้องระบุชื่อให้ตรงกับระบบฐานข้อมูลที่ต้องการติดต่อ

```
try {
    Class.forName ("sun.jdbc.odbc.JdbcOdbcDriver");
} catch (ClassNotFoundException e) {
    System.err.print (e.getMessage ());
}
```

B) ขั้นตอนการติดต่อไปยังระบบฐานข้อมูลปลายทาง

รูปแบบ

```
Connection Instance_Name = DriverManager.getConnection (URL, Login, Password)
```

คลาสดักจับข้อผิดพลาด

SQLException

รูปแบบ URL

jdbc:<subprotocol>:<subname>

เมื่อ

<subprotocol> คือ โพรโตคอลการสื่อสารข้อมูล เพื่อเชื่อมไปยังเครื่องหรือระบบปลายทาง
ซึ่งมีข้อมูลแตกต่างกันไปตามรูปแบบการสื่อสาร เช่น

odbc สำหรับการสื่อสารผ่านระบบ ODBC

dbnet สำหรับการสื่อสารผ่านโปรโตคอล TCP/IP

<subname> คือฐานข้อมูลภายในระบบที่อ้างผ่าน subprotocol หรือแหล่งข้อมูล

ขั้นตอนที่ 2 การส่งคำสั่งเอสคิวแอล (SQL)

เมธอดที่ถูกกำหนดไว้สำหรับส่งคำสั่งเอสคิวแอลไปให้กับฐานข้อมูล ถูกแบ่งตามรูปแบบคำสั่งเอสคิวแอล ได้ 2 แบบ โดยผู้ใช้ต้องเลือกเมธอดที่เหมาะสมกับคำสั่งเอสคิวแอลที่ใช้งานคือ

หากคำสั่งเอสคิวแอลเป็นประเภทดำเนินการ ซึ่งสร้างด้วยคีย์เวิร์ดจำพวก ครีเอท (CREATE), อัลเทอ (ALTER), ดรอป (DROP), อินเซท (INSERT), อัปเดต (UPDATE), ดิลิท (DELETE), ? การทำงานของคำสั่งลักษณะนี้ เป็นการส่งคำสั่งไปยังฐานข้อมูล เพื่อให้เกิดการเปลี่ยนแปลงใดๆในระบบฐานข้อมูล ดังนั้นผู้ใช้คำสั่งเพียงต้องการรับทราบแล้วว่าทำงานสำเร็จหรือไม่เท่านั้น การเรียกใช้คำสั่งเหล่านี้ให้ทำงานจึงต้องใช้งานผ่านเมธอด เอกสคิวอัปเดต (ExecuteUpdate)

หากคำสั่งเอสคิวแอล เป็นประเภทสืบค้นข้อมูล ซึ่งสร้างด้วยคีย์เวิร์ด ซีเลคท (SELECT) เนื่องจากคำสั่งประเภทสืบค้นข้อมูลมีลักษณะพิเศษที่เมื่อระบบฐานข้อมูลได้รับคำสั่งเอสคิวแอล แล้วจะต้องส่งข้อมูล ผลลัพธ์ที่ต้องการกลับมาในรูปแบบของกลุ่มข้อมูลเรคคอร์ด (Record Set) การเรียกใช้งานคำสั่งเอสคิวแอล ประเภทนี้จึงต้องเรียกใช้งานผ่านเมธอดเอกสคิวคิวรี (ExecuteQuery) และต้องมีคลาสมารับผลลัพธ์ที่ได้รับกลับมาในรูปแบบของกลุ่มเรคคอร์ดคือคลาส รีซอลทเซต (ResultSet) เมธอดทั้งสองรูปแบบดังกล่าว สามารถเรียกใช้งานได้จากคลาส Statement และคลาส PreparedStatement อย่างใดอย่างหนึ่ง แต่ทั้งสองคลาสดังกล่าวใช้สำหรับการสร้างคำสั่งเอสคิวแอล ที่แตกต่างกัน คือ

Statement ใช้สำหรับสร้างคำสั่งเอสคิวแอล ที่เป็นข้อมูลคงที่ (ข้อมูลในคำสั่งเอสคิวแอลไม่เปลี่ยนแปลง)

PreparedStatement ใช้สำหรับสร้างคำสั่งเอสคิวแอล ที่สามารถเปลี่ยนแปลงข้อมูลบางส่วนภายในตามพารามิเตอร์ที่ต้องการ

รูปแบบคลาส Statement

int ExecuteUpdate()

สำหรับส่งงานคำสั่ง SQL ประเภทจัดการข้อมูล

ResultSet executeQuery()

สำหรับส่งงานคำสั่ง SQL ประเภทสืบค้นข้อมูล

รูปแบบคลาส PreparedStatement

<code>void setBoolean(int index, boolean x)</code>	กำหนดข้อมูล boolean ให้กับพารามิเตอร์ตำแหน่ง index
<code>void setByte(int index, byte x)</code>	กำหนดข้อมูล byte5 ให้กับพารามิเตอร์ตำแหน่ง index
<code>void setShort(int index, short x)</code>	กำหนดข้อมูล short ให้กับพารามิเตอร์ตำแหน่ง index
<code>void setInt(int index, int x)</code>	กำหนดข้อมูล int ให้กับพารามิเตอร์ตำแหน่ง index
<code>void setLong(int index, long x)</code>	กำหนดข้อมูล long ให้กับพารามิเตอร์ตำแหน่ง index
<code>void setFloat(int index, float x)</code>	กำหนดข้อมูล float ให้กับพารามิเตอร์ตำแหน่ง index
<code>void setDouble(int index, double x)</code>	กำหนดข้อมูล double ให้กับพารามิเตอร์ตำแหน่ง index
<code>void setString(int index, String x)</code>	กำหนดข้อมูล String ให้กับพารามิเตอร์ตำแหน่ง index
<code>void setDate(int index, Date x)</code>	กำหนดข้อมูล Date ให้กับพารามิเตอร์ตำแหน่ง index
<code>void setTime(int index, Time x)</code>	กำหนดข้อมูล Time ให้กับพารามิเตอร์ตำแหน่ง index
<code>void clearParameter()</code>	ลบข้อมูลออกจากพารามิเตอร์ทุกตัว
<code>int executeUpdate()</code>	สำหรับสั่งงานคำสั่ง SQL ประเภทจัดการข้อมูล
<code>ResultSet executeQuery()</code>	สำหรับสั่งงานคำสั่ง SQL ประเภทสืบค้นข้อมูล
<code>void close()</code>	ปิดช่องทางการสื่อสาร

การขยับตัวชี้เพื่ออ้างอิงไปยังแถวที่ต้องการสามารถขยับได้โดยใช้เมธอดของ รีซอลทเซท

ResultSet ดังนี้

- หากต้องการย้ายไปที่ข้อมูลแถวแรกใช้เมธอด **first()**
- หากต้องการย้ายไปที่ข้อมูลแถวสุดท้ายใช้เมธอด **last()**
- หากต้องการย้ายไปที่ข้อมูลแถวถัดไปจากตำแหน่งปัจจุบันใช้เมธอด **next()**
- หากต้องการย้ายไปที่ข้อมูลแถวก่อนหน้าจากตำแหน่งปัจจุบันใช้เมธอด **previous()**
- หากต้องการย้ายไปที่ข้อมูลตามแถวตัวเลขที่ต้องการใช้เมธอด **absolute(แถวที่ต้องการ)**

ในกลุ่มเมธอดที่ใช้ในการตรวจสอบตำแหน่งแถวว่าอยู่แถวใด มีเมธอดในกลุ่มที่เกี่ยวข้องดังนี้

- หากต้องการทราบว่าขณะนี้ตัวชี้อยู่ที่ตำแหน่งแถวใดให้ใช้เมธอด **getRow()**
- หากต้องการทราบว่าขณะนี้ตัวชี้อยู่ตำแหน่ง Before First หรือไม่ให้ใช้เมธอด **isBeforeFirst()**
- หากต้องการทราบว่าขณะนี้ตัวชี้อยู่ตำแหน่ง First หรือไม่ให้ใช้เมธอด **isFirst()**
- หากต้องการทราบว่าขณะนี้ตัวชี้อยู่ตำแหน่ง After Last หรือไม่ให้ใช้เมธอด **isAfterLast()**
- หากต้องการทราบว่าขณะนี้ตัวชี้อยู่ตำแหน่ง Last หรือไม่ให้ใช้เมธอด **isLast()**

ขั้นตอนที่ 3 การแสดงผลลัพธ์สู่ผู้ใช้

ผลลัพธ์ในชุดรีซอลต์เซต (**ResultSet**) มีโครงสร้างคล้ายกับข้อมูลในเทเบิล คือแบบออกเป็นแนวแถวเรียกว่า **Row** หรือ เรคอร์ด และแนวคอลัมน์เรียกว่า **Column** หรือ ฟิลด์ ดังนั้นหลักการดึงข้อมูลก็ต้องระบุไรต์ให้ชัดเจนก่อนแล้วจึงค่อยระบุคอลัมน์ที่ต้องการ

รูปแบบคลาส ResultSet

<code>boolean getBoolean(String ColumnName)</code>	อ่านข้อมูล boolean ในฟิลด์ ColumnName
<code>byte getByte(String ColumnName)</code>	อ่านข้อมูล byte ในฟิลด์ ColumnName
<code>short getShort(String ColumnName)</code>	อ่านข้อมูล short ในฟิลด์ ColumnName
<code>int getInt(String ColumnName)</code>	อ่านข้อมูล int ในฟิลด์ ColumnName
<code>long getLong(String ColumnName)</code>	อ่านข้อมูล long ในฟิลด์ ColumnName
<code>float getFloat(String ColumnName)</code>	อ่านข้อมูล float ในฟิลด์ ColumnName
<code>double getDouble(String ColumnName)</code>	อ่านข้อมูล double ในฟิลด์ ColumnName
<code>String getString(String ColumnName)</code>	อ่านข้อมูล String ในฟิลด์ ColumnName
<code>Date getDate(String ColumnName)</code>	อ่านข้อมูล Date ในฟิลด์ ColumnName
<code>Time getTime(String ColumnName)</code>	อ่านข้อมูล Time ในฟิลด์ ColumnName

ขั้นตอนที่ 4 การยกเลิกการติดต่อ

สำหรับในขั้นตอนยกเลิกการติดต่อนี้เป็นขั้นตอนที่ง่ายและไม่ยุ่งยากในการทำงาน หากเมื่อพิจารณาจากการดำเนินการติดต่อข้อมูล มีอยู่ 2 ส่วนด้วยกัน ส่วนแรกคือช่องทางการติดต่อที่สร้างมาจากเมธอด `getConnection` และส่วนที่สองคือชุดคำสั่ง **ResultSet** ที่ได้จากเมธอด `executeQuery`

```

Connection con = DriverManager.getConnection ( url, "myLogin", "myPassword");
Statement stmt = con.createStatement();
ResultSet rs = stmt.executeQuery("SELECT * FROM Products")
?
?
?
rs.close();
con.close();

```